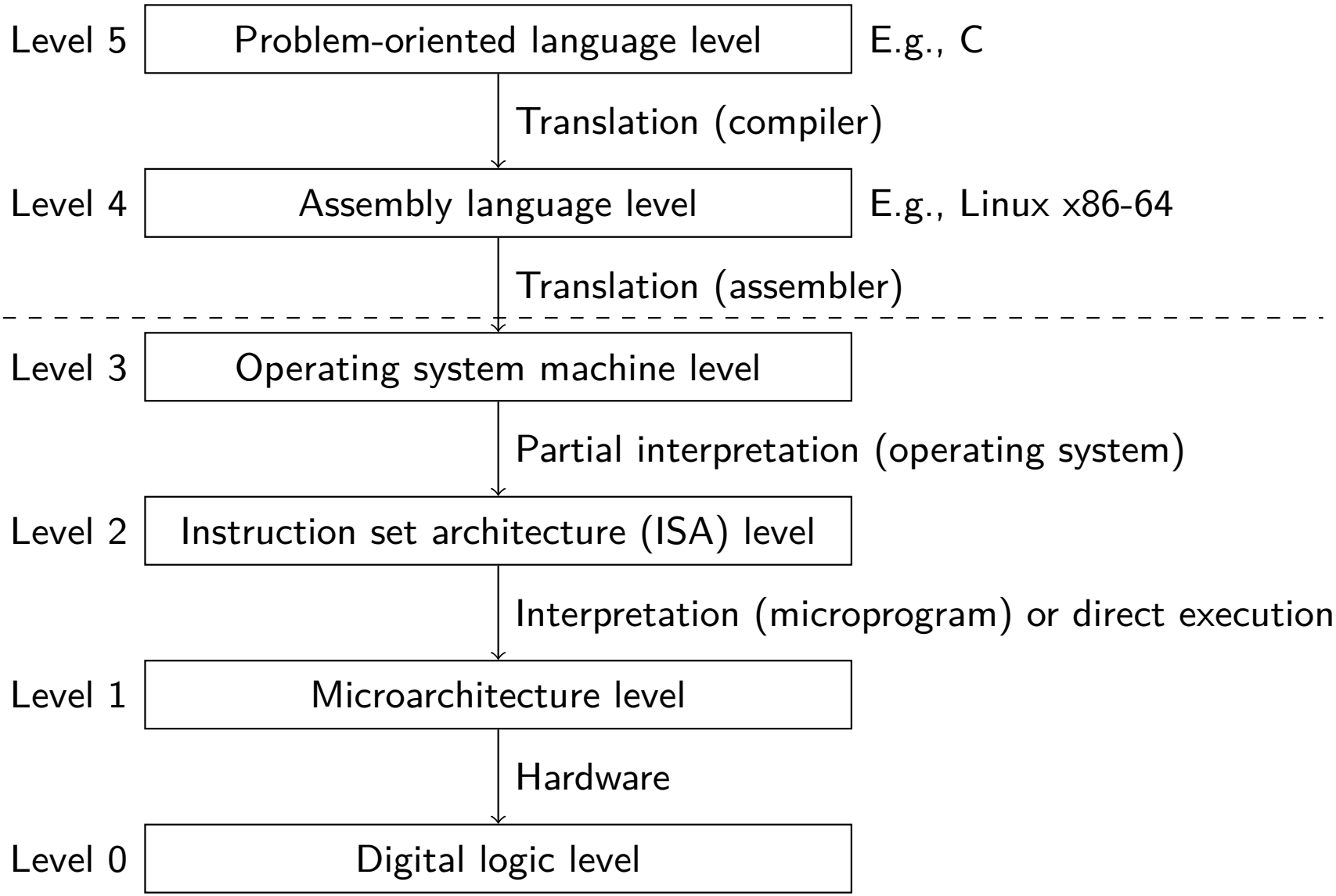
DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2026

Introduction to C

- Introduction
- Preprocessor
- Basic Types
- Constants
- Operators
- Control Flow
- Arrays and Pointers

Where Are We?


Level 5
Problem-oriented language level
E.g., C
Translation (compiler)
Level 4
Assembly language level
E.g., Linux x86-64
Translation (assembler)
Level 3
Operating system machine level
Partial interpretation (operating system)
Level 2
Instruction set architecture (ISA) level
Interpretation (microprogram) or direct execution
Level 1
Microarchitecture level
Hardware
Level 0
Digital logic level

What is System(s) Programming?

- Application programming: the product is used by end users, e.g., MS Word.
- System programming: the product is a intended more for other specialists and other software.
  (though, the distinction is not that black and white)
- Performance constrained.
- Minimizing runtime overhead.
- Possible to have direct control over memory.
- Usually without garbage collection.
- Compiled to native code.
- Example products: operating systems, compilers, scientific software, industrial software, game engines, microcontrollers, embedded software.
- Current common languages: C, C++, Rust, (D, Swift, Go)
- Those languages *facilitate* system programming, but they can of course be used for application programming as well.

What is C?

- Developed by Dennis Ritchie at AT&T Bell Laboratories in 1972.
- Based on the B programming language.
- Designed for (re)implementing and programming UNIX.
- First for the 16-bit DEC PDP-11.
- Later ported to other systems.
- Abstracts the details of assembly,
- but easily maps back to assembly.
  ("C is portable assembly")
- A rather low-level language, not many fancy abstractions.
- Still widely used.
- Many subsequent languages are based on C in some way
  (as a minimum the syntax is borrowed).
- Almost all C code is valid C++ code
  (and does the same thing).

History of C

- 1972 C is invented.
- 1978 "The C Programming Language", Brian Kernighan and Dennis Ritchie is published. Aka. K&R C.
- 1989 ANSI publishes a standard for C.
  Aka. C89, ANSI C, Standard C.
  Incorporates popular extensions and features from C++.
- 1990 ISO adopts the ANSI standard. Aka. C90 (same as C89).
- 1999 The C99 standard. Mostly backwards compatible with C89.
- 2011 The C11 standard. Many new features, e.g., multi-threading, atomics, type-generic macros.
- 2018-24 The C17 and C23 standards.
- 202y The upcoming C2y standard.

History of C

1972 C is invented.

1978 "The C Programming Language", Brian Kernighan and Dennis Ritchie is published. Aka. K&R C.

1989 ANSI publishes a standard for C.
Aka. C89, ANSI C, Standard C.
Incorporates popular extensions and features from C++.

1990 ISO adopts the ANSI standard. Aka. C90 (same as C89).

1999 The C99 standard. Mostly backwards compatible with C89.

2011 The C11 standard. Many new features, e.g., multi-threading, atomics, type-generic macros.

2018-24 The C17 and C23 standards.

202y The upcoming C2y standard.

Consequences:

- There is no single company that decides what is correct.
- The standard defines what a C program is.
- Each compiler may implement the standard to varying degree.
- If you write something not C, the result may be surprising.

Relation to Other Languages

C has directly and indirectly influenced many languages, e.g.,

- C++ (also from Bell Labs (1979) and now an ISO standard).
  - Java (1995, owned by Oracle)
  - C♯ (2002, owned by Microsoft)
- Objective C, Perl, Python, JavaScript, ...

Features (or lack thereof):

- Structured programming.
- A very small and simple language.[1]
- No support for object-oriented programming.*
- No exceptions.
- No function overloading.(**)
- No range-checking on array indexing.***
- No garbage collection, must explicitly deallocate memory.
- Not that much type safety.
- Easy to interact with other languages!

Popular myth: if you want performance, go to C.

[1] But not a simple language to use.

It may be easier than assembly, but . . .

Inherently unsafe:

- No range checking on array indexing.
- Easy to (accidentally) subvert the type system.
- No runtime checks (however: sanitizers!).

Sometimes very inconvenient:

- The compiler may not detect many errors compilers for other languages may find. Why? Because of efficiency.
- You must manually handle all resources, notably memory.

Handle with care:

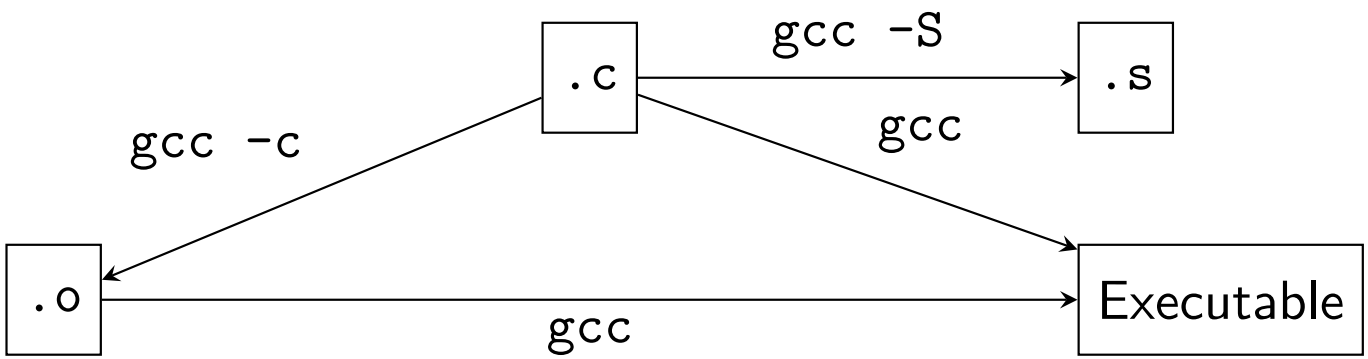
- It may seem to work, but is it correct? "Undefined behaviour".
- Use tools to help you: GDB, Valgrind, Godbolt, sanitizers, . . .

Summary

- C without libraries can do almost nothing.
- C will not get in your way when you want efficiency.
- C will not protect you when you make a mistake.
- The programmer is expected to know what they are doing.
- Can be thought of as high-level, portable assembly.

Creating a C Program

- Source files should end in "`.c`".
  (Header files should end in "`.h`")
- There are many compilers, we will use the C compiler in the GNU Compiler Collection, `gcc`.




```
> gcc -o myProg some.c code.c
```

or

```
> gcc -c -o some.o some.c
> gcc -c -o code.o code.c
> gcc -o myProg some.o code.o
```

Compiler Options

There are **many** options for `gcc`, but some important ones are:

- `-Wall`: enable "all" warnings.
- `-Wextra`: enable additional warnings, on top of `-Wall`
- `-g`: enable debug symbols.
  (useful when using sanitizers/GDB).
- `-O0`, `-O1`, `-O2`, `-O3`: set the optimisation level (default is 0).
- `-std=c99`: compile against a specific C standard.
- `-fsanitize=address`
  `-fsanitize=leak`
  `-fsanitize=undefined`
  See next slide.

Don't be brave and lazy, as default use:

```
-Wall -Wextra -g -fsanitize=address,leak,undefined
```

Are you expected to type all those crazy options all the time?

No, we can/should use a Makefile.

Debugging

Sanitizers: (highly recommended to use)

- Make the compiler insert runtime checks for certain errors. This may slow down the program somewhat.
- There are different types of sanitizers, not all compatible with each other.
- But, compiling with the following should work:
  `-fsanitize=address,leak,undefined`
- Compile with `-g` to get better error messages.

Valgrind: (not compatible with sanitizers)

- Run the program through a kind of virtual machine.
- Detects memory problems (like ASan + LeakSan but much slower).
- `valgrind -leak-check=full -track-origins=yes ./myProg arg1 arg2`

GDB: (useful also with sanitizers or Valgrind)

- Similar to your usage with assembly code.
- Many new possibilities for break points.
- Use "`step`" to go to next source line.
- Use "`stepi`" to go to next assembly instruction.

Hello World!

```c
#include <stdio.h> // A preprocessor directive.

int main(void) { // The entry point.
    // Print formatted output to stdout.
    printf("Hello World!\n");
    // Set the exit code.
    return 0;
}
```

- The program starts in "`main`". It must be declared as either of
  `int main(void)`
  `int main(int argc, char **argv)`
- Write with "`printf`" to stdout, or "`fprintf`" to any file.
- We must explicitly specify the exit code:
  return from "`main`" or using "`exit(yourExitCode)`".
- Lines starting with "`#`" are preprocessor directives (more later).
- `#include <somefile.h>` is how we access library functionality.
- `https://en.cppreference.com` and "`man`" are your friends.

Function Prototypes

- C was invented long ago.
- The compiler is not required to remember the complete input.
- Imagine reading the C code and generating assembly on the fly.
- The compiler must be told about functions/variables textually before they are used. (see `after.c` and `before.c`)
- A definition: the implementation of a function / the memory for a variable.
- There can be only one definition of each entity.*
- A declaration: a bit of information about a function/variable, enough to use it. (see `afterDecl.c`)
- A function declaration is often called a function prototype.
- Library header files usually contain declarations of functions already compiled.

* Well, you *can* make multiple definitions in different files, but it's UB, so it's no longer C code, so all bets are off.

The `main` function:

- The entry point for a program.
- `int main(void);`, command line arguments not available.
- `int main(int argc, char **argv);` (more later)

Functions without parameters:

- `int f(void);`, a function taking exactly 0 arguments.
- `int f();`, a function taking an unknown amount of arguments.
- This is a C-specific weirdness, due to history.
- Don't do prototypes with empty parenthesis, unless you are sure of what you are doing.

- Introduction
- Preprocessor
- Basic Types
- Constants
- Operators
- Control Flow
- Arrays and Pointers

What is the Preprocessor?

- Runs before the real compilation.
- Controlled by lines starting with "#".
- Text substitution through macros.
- Conditional removal of content.
- Including the contents of other files.
- Use "`gcc -E file.c`" to stop after preprocessing.

`https://en.cppreference.com/w/c/language/translation_phases`

`https://en.cppreference.com/w/c/preprocessor`

Inclusion

`#include "a/b/file.h"`

searches for the file in the current folder first.

`#include <a/b/file.h>`

searches some list of folders.

- `gcc -I`*some/path/to/headers* `-I`*and/another/path* ...
  Adds extra search folders.
- Which folders are being searched? `gcc -xc -E -v -`
- Simplified, it copy-pastes the content of the specified file.
- Usually used to read declarations from other files.
- Usually those files end with "`.h`", and are called header files.
  They are still C files, but special by universal convention.
- The standard library has many useful headers, e.g.,
  `stdio.h`, `math.h`, `assert.h`, `ctype.h`,
  `stdbool.h`, `stdlib.h`, `string.h`, `time.h`

`https://en.cppreference.com/w/c/header`
`https://en.cppreference.com/w/c/preprocessor/include`

Macros

Performs text-substitution on the tokenized input.
Multiple forms, e.g.:

- `#define WITH_MY_DEBUG_CODE`
  Define a new name in the preprocessor, with empty substitution.
- `#define PI 3.14`
  Define a substitution of "`PI`" with "`3.14`".
- `#define CIRCLE_AREA(radius) PI * radius * radius`
  A function-like substitution, which must be called with an argument.
- `#undef PI` or `#undef CIRCLE_AREA`
  Forget a macro.
- Use "`gcc -Dvar`" or "`gcc -Dvar=val`" to make `#define`s on the command line.

`https://en.cppreference.com/w/c/preprocessor/replace`

Macros

Be careful, "`radius`" is not a C variable:

```
CIRCLE_AREA(expensiveFunction())
```

expands to

```
3.14 * expensiveFunction() * expensiveFunction()
```

and

```
CIRCLE_AREA(a + b)
```

expands to

```
3.14 * a + b * a + b
```

and

```
CIRCLE_AREA((a + b))
```

expands to

```
3.14 * (a + b) * (a + b)
```

---

http://ioccc.org/2001/herrmann1.hint

https://www.boost.org/doc/libs/release/libs/preprocessor/

Conditional Content

`#if`, `#ifdef`, `#ifndef`, `#else`, `#elif`, `#endif`

- Depending on macros, keep or skip lines.
- Remember, there is no C code at this stage, only text.
- Used all the time used to make header inclusion guards (more later).
- Other common uses:
  - Enabling/disabling debug code.
  - Enabling/disabling code depending on the environment (Windows/Linux/Android, GCC/Clang, MyLaptop/IMADA, . . .).

`https://en.cppreference.com/w/c/preprocessor/conditional`

Small Example

```
...
#ifdef DEBUG
   printf("Debug: blah blah");
#endif
...
```

"gcc -DDEBUG ..." will contain the printing.
"gcc ..." will not.

- Introduction
- Preprocessor
- Basic Types
- Constants
- Operators
- Control Flow
- Arrays and Pointers

Types

- C is statically typed, all variables and functions must have types specified (as in Java, Python is dynamically typed).
- The special type `void` is used for "no return type" (as in Java, Python use "None") and "no parameters" (a C-oddity).
- Most basic types (built-in types) are direct hardware abstractions:
  `https://en.cppreference.com/w/c/language/type`
  - Integer/floating point types of different sizes.
  - Integer types with different interpretations (signed/unsigned).
- The "`_Bool`"/"`bool`" type was a C99 addition.
  Was made a proper type with `true` and `false` constants in C23.
- Be careful with signed vs. unsigned, they are mathematically different (more later).
- "`char`" is usually the type used for representing a text character.
- "`unsigned char`" is usually the type used for a raw memory.

Width of Types

The standard doesn't dictated the exact width, only lower bounds.
E.g.,

- `short`, at least 16 bits.
- `int`, at least 16 bits (probably 32 on our Linux machines).
- `long`, at least 32 bits (probably 64 on our Linux machines).
- `long long`, at least 64 bits (probably 64 on our Linux machines).
- `float`, single precision float, usually IEEE-754 standard.
- `double`, double precision float, usually IEEE-754 standard.
- `long double`, extended precision float (perhaps IEEE-754 standard).

Actually, C counts in units of `char` (which has `CHAR_BIT` number of bits).
On most systems `CHAR_BIT` is 8.
Guaranteed:

$1 =$ `sizeof(char)` $\leq$ `sizeof(short)`
$\leq$ `sizeof(int)` $\leq$ `sizeof(long)` $\leq$ `sizeof(long long)`

---

https://en.cppreference.com/w/c/language/arithmetic_types

Size of Data Types on Different Systems

`short`	`int`	`long`	pointer	`long long`	Label	Examples
—	16	—	16	—	IP16	PDP-11 Unix (1973)
16	16	32	16	—	IP16L32	PDP-11 Unix (1977)
16	16	32	32	—	I16LP32	Macintosh, Windows
16	32	32	32	—	ILP32	IBM 370, VAX Unix
16	32	32	32	64	ILP32LL64	Win32, Unix (1990s)
16	32	32	64	64	IL32LLP64	Win64
16	32	64	64	64	I32LP64	Unix (most of them)
16	64	64	64	64	ILP64	HAL
64	64	64	64	64	SILP64	UNICOS

Reminder: different systems have different byte orderings

- Big endian: most significant byte on lowest address, e.g., PowerPC.
  Also the order used when transferring data over networks.
- Little endian: least significant bytes on lowest address, e.g., all x86 systems.

`https://queue.acm.org/detail.cfm?id=1165766`

Constants

Type Class	Examples

Integers	`int x = 3;`
	`long x = 3L;`
	`unsigned long x = 3ul;`
	`int x = 0xA; //hexadecimal`
	`int x = 012; //octal`

Floats	`float x = 3.14f;`
	`double x = 3.14;`
	`long double x = 3.14L;`

Character	`char x = 'A';`
	`char x = '\x5A'; //hexadecimal`
	`char x = '\101'; //octal`

Remember: (`signed`/`unsigned`) `char` are still just integers.
We just happen to often visualise `char` graphically.

---

https://en.cppreference.com/w/c/language/integer_constant
https://en.cppreference.com/w/c/language/character_constant
https://en.cppreference.com/w/c/language/floating_constant

- Introduction
- Preprocessor
- Basic Types
- Constants
- Operators
- Control Flow
- Arrays and Pointers

Arithmetic Operators

Expression	Notes
`+a`	
`-a`	
`a + b`	
`a - b`	
`a * b`	
`a / b`	
`a % b`	
`~a`	Bitwise complement
`a & b`	Bitwise AND
`a | b`	Bitwise OR
`a ^ b`	Bitwise XOR
`a << b`	Logical left-shift, be careful with signed integers
`a >> b`	Logical right-shift, be careful with signed integers

`https://en.cppreference.com/w/c/language/expressions#Operators`

`https://en.cppreference.com/w/c/language/operator_arithmetic`

Assignment Operators

Expressions			
`a = b`	`a *= b`	`a &= b`	`a <<= b`
`a += b`	`a /= b`	`a |= b`	`a >>= b`
`a -= b`	`a %= b`	`a ^= b`	

- The expression has the value of the left-hand side after assignment, so "`(a = 42) == 42`".
- Chaining: "`a = b = c = d`" is valid, meaning "`a = (b = (c = d))`".
- The arithmetic operator in compound assignments doesn't influence the associativity:
  "`a *= b += c = d`" still means "`a *= (b += (c = d))`"
  (it's probably not a good idea to chain compound assignments)

`https://en.cppreference.com/w/c/language/operator_assignment`

Increment Operators

Expressions
`++a`
`--a`
`a++`
`a--`

- Don't modify the same variable in the same expression: "`i = a++ * a++ - a++`", has undefined behaviour.
- So does "`i = i++`".

`https://en.cppreference.com/w/c/language/operator_incdec`

Logical and Relational Operators

- `bool`, `true`, and `false` was made proper entities in C23.
- Before they were macros, defined in `#include <stdbool.h>`.
- Therefore, `0` is still considered `false`, everything else is `true` (e.g., `0.5` is `true`).

Expressions		
`!a`	`a == b`	`a > b`
`a && b`	`a != b`	`a <= b`
`a || b`	`a < b`	`a >= b`

https://en.cppreference.com/w/c/language/arithmetic_types
https://en.cppreference.com/w/c/language/operator_logical
https://en.cppreference.com/w/c/language/operator_comparison

Other Operators

Expression	Notes
`f(...)`	Function call.
`a, b`	Evaluate `a`, discard the result, evaluate and return `b`.
`(type)a`	Cast the result of `a` to type `type` (more later).
`a ? b : c`	Conditional operator.
`sizeof(a)`	Size (in bytes) of an expression or a type `a`.
`_Alignof(type)`	New in C11, return the alignment requirement.

- The evaluation of function arguments happen in no particular order, and may be interleaved.
- `f(g(), h())`, ok, unspecified if `g` or `h` is called first.
- `f(i, i++)`, undefined behaviour.

(there are also member access operators, more later)

---

https://en.cppreference.com/w/c/language/operator_other

https://en.cppreference.com/w/c/language/eval_order

Type Conversions

- Implicit conversions are very complicated, a bit of a mess, and has many pitfalls.
- When variables are promoted to higher precision, the value is preserved.
- This is automatically done by the compiler for mixed data type expressions:

```
int i = 5;
double f = i + 3.14; // i is promoted to double,
// as if we wrote
double f = (double)i + 3.14;
```

- Very simplified: the narrower type is promoted to the wider type.
- Integer values are similarly promoted, but it's complicated due to `signed`/`unsigned`.
- Conversion from wide to narrow types are possible, values may change.

https://en.cppreference.com/w/c/language/conversion

- Introduction
- Preprocessor
- Basic Types
- Constants
- Operators
- Control Flow
- Arrays and Pointers

Basic and Compound Statements

- Basically as in Java.
- Compared to Python: indentation level and line breaks are ignored.
- Simple statements ends with semicolon:

```
x = foo(y);
```

- Curly braces combine statements into a block, and creates a new scope:

```
int a = 42;
{
    int temp = a + 1;
    a = foo(temp);
}
// temp is no longer in scope
```

- We can shadow a variable in an inner block by reusing the name (often a bad idea for readability).
- Blocks can be nested.

https://en.cppreference.com/w/c/language/statements

If Statement

- As in Java.
- "else-if" is written `else if`

Examples:

```
if(a != b)
   x += y // single statement doesn't need {...}
else
   q = 1; // same for else/else-if statements

if(z > 60) q = 2; // and line breaks are only for the reader

if(a < b) { // but they are needed for multiple statements
   x -= y;
   z = 42;
}
```

https://en.cppreference.com/w/c/language/if

Switch Statement

- Not as in Java.
- Only integer, character, and enumeration types can be given.
- A special case of `if` constructs.

```
switch(c) {
case 5: // if(c == 5), do this part
   x = 2;
   break; // jump out of the switch
case 42:
   y = -1;
   // there is no 'break', so fall through to the next case
case 60:
   x = 3; // execute both for c == 42 and c == 60
   break;
default: // optional, if c is none of the mentioned values
   x = -30;
   break;
}
```

https://en.cppreference.com/w/c/language/switch

Iteration Statements

- Basically the same as in Java.
- `while` loop (as in Python, but slightly different syntax)
- `do-while` loop (`do { ... } while(condition);`)
- `for` loop

```
for(int i = 0; i != 42; ++i) {
    stuff();
}
```

means

```
{
    int i = 0;
    while(i != 42) {
        stuff();
        ++i;
    }
}
```

- `break` and `continue` statements.

---

https://en.cppreference.com/w/c/language/while
https://en.cppreference.com/w/c/language/do
https://en.cppreference.com/w/c/language/for

Goto Statement

- An unconditional jump to an arbitrary point within the same function.
- The location is specified with a label, similar to labels in x86 assembly code.
- A statement is labelled as in x86 assembly code:

```
start:
{
    if(something)
        goto outside;
    // do stuff
    goto start;
}
end:; // an empty statement
```

- another example

```
goto lab1; // jump past the initialization
    int n = 5;
lab1:;
// n is in scope, but is uninitialized,
// as if declared by 'int n;'
```

https://en.cppreference.com/w/c/language/goto

goto for Error Handling

- C does not have exceptions.
- goto enables us to jump conveniently.

```
int okFlag = 1;
for(...) {
   for(<init>; okFlag && <cond>; <iter>) {
      if(<error condition>) {
         okFlag = 0;
         break;
      }
   }
   if(!okFlag) break;
}
<error handling code>
```

```
for(...) {
   for(...) {
      if(<error condition>)
         goto error;
   }
}
...
error:
   <error handling code>
```

Spaghetti Code

Dijkstra. *Go To Statement Considered Harmful*. Communications of the ACM, **11(3)**, 1968

- Excess use of `goto` creates spaghetti code.
- Using `goto` makes code harder to read and debug.
- Any code that uses `goto` can be written without using one.
- Also, be very careful of unexpected side-effects . . .



[https://xkcd.com/292/]

- Introduction
- Preprocessor
- Basic Types
- Constants
- Operators
- Control Flow
- Arrays and Pointers

Array Basics

- Remember the assembler directives `.space`, `.int`, `.float`, ...?
- An global array definition in C can be thought as an abstraction.
- When arrays are defined in functions, they are on the stack.
- Declaration, array of 10 `int`s: `int arr[10];`
  Each element has indeterminate value, remember to initialise.
- Indexing (0-based): `arr[5]`
- C declarations are written as how the variables can be used:
  "`arr` is something we can index into, up to 10,
  and then we get an `int`"
- Array size must* be specified at compile time.

`https://en.cppreference.com/w/c/language/array`

* there are variable-length arrays (special for C), which we won't discuss.

Array Initialization

```c
int arr1[5] = {1, 2, 3, 4, 5};
// Shorthand, the compiler inserts the size
// based on the initializer.
// Does exactly the same as above.
int arr2[] = {1, 2, 3, 4, 5};
// Specify a prefix, fill with 0:
int arr3[5] = {1, 2, 3};
// gives [1, 2, 3, 0, 0]
```

There are special features for character arrays, i.e., strings. (more later)

https://en.cppreference.com/w/c/language/array_initialization

Array Copying

Arrays do not support copying:

```
int arr1[42];
...
int arr2[42];
arr2 = arr1; // error
```

Therefore arrays can not be given as function arguments by value.

```
void f(int a[10]) {
    a[0] = 42;
}

int g() {
    int arr[10] = {0};
    f(arr);
    return arr[0]; // 42
}
```

Pointer Basics

- A pointer can be thought of as the address of some piece of memory.
- The constant `NULL` is the value for pointing to nothing.
- Declaration: `int *p;`
- Dereference, get the thing it points to: `int i = *p;`
- So, `int *p;` means "when `p` is dereferenced we get an `int`".
- What about `int **pp;`?
  When `pp` is dereferenced we get an `int*`, which can be dereferenced to get an `int`.
- "Address of", the inverse of dereference:

```
int i = 42;
int *p = &i;
int **pp = &p;
int j = **pp; // 42
```

---

https://en.cppreference.com/w/c/language/operator_member_access

Pointer Arithmetic and Arrays

Array variables usually decay to pointers:

```
int arr[10];
int *first = arr; // decay
```

Pointers and integers may be added:

```
int *p = first + 5;
```

but **only** if the resulting pointer is within the same array,
or 1 past the end:

```
int *last = first + 10; // 1-past-the-end
```

Pointers from the same object may be subtracted:

```
int len = last - first;
```

Array indexing is just a shorthand for pointer arithmetic and dereference: `arr[5]` means `*(arr + 5)`
so does `5[arr]` (but don't do this)

Arrays As Function Arguments

Arrays also decays to pointers in function arguments:

```
void f(int *arr);
void g(int arr[]); // same as f
void h(int arr[42]); // still the same as f
// the 42 is not checked, the compiler ignores it
```

So, the `main` function could be declared as:

```
int main(void)
int main(int argc, char **argv)
int main(int argc, char *argv[])
```

or probably also like

```
int main(int argc, char *argv[42])
```

(but don't do this)

Array/Pointer Example

```
void fill(int *arr, int len, int val) {
   for(int i = 0; i != len; ++i)
      arr[i] = val;
}
```

```
void fill(int *first, int *last, int val) {
   for(; first != last; ++first)
      *first = val;
}
```

Pointer to Array, Array of Pointers

`char *foo[42];`, what does it mean?

Remember: the declaration is like how it will be used.

1. The array subscription has higher precedence than dereference.
2. So `foo` is an array of size 42.
3. When indexing into it, we get a `char*`.
4. So `foo` is an array of 42 pointers.

Pointer to Array, Array of Pointers

`char *foo[42];`, what does it mean?

Remember: the declaration is like how it will be used.

1. The array subscription has higher precedence than dereference.
2. So `foo` is an array of size 42.
3. When indexing into it, we get a `char*`.
4. So `foo` is an array of 42 pointers.

How does a pointer to an array look?

1. Let it be called `bar`.
2. It should be a pointer so we start with `*bar`.
3. To go against the precedence, use parenthesis (as always).
4. So: `(*bar)[42]`
5. And with the element type: `char (*bar)[42]`.
6. A pointer to an array of 42 characters.

Note, usually one just uses a pointer instead of pointer to array.

Strings

- A sequence of non-zero bytes followed by a 0-byte.
- E.g., `char str[] = {'H', 'e', 'l', 'l', 'o', '\0'};`
- String literals have an implicit 0-byte: `"Hello"`
- Character arrays can be initialised from literals:
  `char str[] = "Hello";`
- String literals can also decay to pointers:
  `char *p = "Hello";`
- Be careful, string literals may not be modified:
  `p[1] = 'a';` has undefined behaviour
  `str[1] = 'a';` is ok, not a string literal modification
- Adjacent string literals are concatenated at compile time:
  `"Hel" "lo"`

---

https://en.cppreference.com/w/c/language/string_literal

https://en.cppreference.com/w/c/string